

Assignment 4: Distributed-Memory Programming with MPI

Jinye Gong
jinyeg@kth.se

Weiyi Lyu
weiyil@kth.se

2026-05-20

Contributions

This assignment was completed jointly by both team members.

- Weiyi Lyu:** Dardel environment setup (`edu26.dd2356`), directory structure, Slurm scripts, and primary runs on Dardel; Exercise 1 implementation and strong-scaling experiments; report structure and MPI sections.
- Jinye Gong:** School-cluster experiments, Exercise 2 and Exercise 3 implementations, correctness validation and Python visualizations, weak-scaling / efficiency measurements, and final report integration.

Both members contributed to algorithm design, result verification, plotting, and report writing.

AI Assistance Statement

AI tools were used for Markdown organization, section structuring, and language polishing.

All implementations, command execution, performance measurements, result collection, plotting, and technical conclusions were performed and verified by the authors.

Screenshot Evidence Index

Screenshots (code, compilation, largest-scale run) are under `Artifacts/screenshots/school/` and `Artifacts/screenshots/dardel/` (Exercises 1–2 on Dardel; Exercise 3 school only per assignment).

Exercise	School	Dardel
Ex1 wave	<code>ex1_*.png</code>	<code>ex1_*.png</code>
Ex2 row sum	<code>ex2_*.png</code>	<code>ex2_*.png</code>
Ex3 GoL	<code>ex3_*.png</code>	—

Score-P (Dardel, Ex1–2, np=16): completed with `score-p/9.4-cpeCray-24.11`; metrics reported in §7 of Exercises 1–2.

Exercise 1 - 1D Halo Exchange in a Wave Equation Simulation

1. Implementation

We parallelized the 1D explicit wave solver in `Exercise 1/wave_mpi.c` using **1D domain decomposition** and **point-to-point halo exchange**.

Parallelization strategy

- Split the global grid of size `N` across `P` MPI processes.
 - Local size: `local_n = N / P`, remainder distributed to the first `N % P` ranks.
 - Each rank owns contiguous indices `[start, end)` in global coordinates.
- Allocate local arrays `u`, `u_prev`, `u_next` with **two ghost cells** (left and right).
- Each time step:
 - Exchange ghost values with neighbors using `MPI_Send` / `MPI_Recv` (or equivalent).
 - Left boundary: rank `0` uses a fixed boundary value; right boundary: rank `P-1` similarly.
 - Update interior points `i = 1 .. local_n` with the finite-difference stencil.
- Timing: `MPI_Wtime()` around the main loop, preceded by `MPI_Barrier(MPI_COMM_WORLD)`.
- I/O: only rank `0` writes `wave_output_*.txt` after gathering the global field (for correctness plots). Disable I/O when measuring scalability.

Core halo exchange pattern (conceptual):

```
/* After local compute setup, each step: */
if (rank > 0) {
    MPI_Send(&u[1], 1, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD);
    MPI_Recv(&u[0], 1, MPI_DOUBLE, rank - 1, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
}
if (rank < nprocs - 1) {
    MPI_Send(&u[local_n], 1, MPI_DOUBLE, rank + 1, 1, MPI_COMM_WORLD);
    MPI_Recv(&u[local_n + 1], 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
}
```

Design notes:

- Ghost cells hold neighbor values required by the 3-point spatial stencil.
- Synchronization per step ensures all ranks have consistent halos before updating `u_next`.
- Strong scaling keeps **global `N` fixed** while increasing `P`.

Parameters used for experiments

Setting	Correctness / plots	Scalability runs
<code>N</code>	1000 (default)	e.g. 100000 (increase for scaling)
<code>STEPS</code>	100	e.g. 1000

Setting	Correctness / plots	Scalability runs
I/O	every 10 steps	disabled

2. Build and run commands

School cluster

```
cd ~/dd2356_a4/Exe1
mpicc -O3 -o wave_mpi wave_mpi.c -lm
chmod +x run_school_ex1.sh run_scaling_strong.sh
./run_school_ex1.sh
```

Compile command: `mpicc -O3 -o wave_mpi wave_mpi.c -lm`

Run command (scaling): `mpirun -np <np> ./wave_mpi` with `WAVE_N=200000 WAVE_STEPS=500 WAVE_IO=0`

Dardel

```
cd ~/DD2356/assignment4
source scripts/dardel_setup_env.sh
cd "Exercise 1"
cc -O3 -o wave_mpi wave_mpi.c -lm

# Interactive (max 4 ranks per node on one node)
salloc -A edu26.dd2356 -N 1 -t 00:30:00 --ntasks-per-node=4
srun -n 4 ./wave_mpi

# Batch strong scaling at np=16 (4 nodes x 4 tasks/node)
cd ~/DD2356/assignment4
mkdir -p logs
sbatch scripts/dardel_run_mpi.slurm 1 16
```

Compile command for report (Dardel): `cc -O3 -o wave_mpi wave_mpi.c -lm` after `module load PDC/24.11 PrgEnv-gnu/8.6.0 cray-mpich/8.1.31`.

Run command for report (Dardel): `srun -n <np> ./wave_mpi` inside an allocated job.

3. Correctness validation

We compared wave outputs from:

1. **Serial baseline** (single-process run or serial reference code).
2. **Parallel MPI run** on the school cluster.

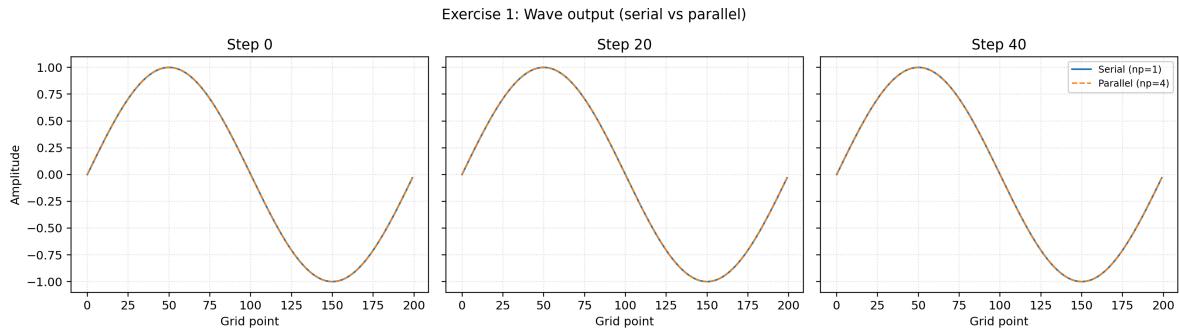
Validation approach:

- Plot `wave_output_*.txt` with `plot_wave.py` and overlay serial vs parallel curves at the same time steps.
- Check max absolute difference between serial and parallel gathered fields (should be near machine epsilon if algorithm matches).

Correctness check (school cluster, `wave_output_0.txt`):

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$ head -5 wave_output_0.txt
0
0.0062831439407541748
0.012566039833744033
0.018848439640997763
0.025130095344128165
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$
```

Serial vs parallel wave visualization (same setup, `N=200`, `STEPS=50`, `I/O=1`; serial = `np=1`, parallel = `np=4`):



School cluster screenshots (code / compile / run / scaling):

```
static void exchange_halos(int rank, int nprocs, int local_n, double *u) {
    MPI_Status status;

    if (rank > 0) {
        MPI_Sendrecv(&u[1], 1, MPI_DOUBLE, rank - 1, 10,
                    &u[0], 1, MPI_DOUBLE, rank - 1, 11,
                    MPI_COMM_WORLD, &status);
    }
    if (rank < nprocs - 1) {
        MPI_Sendrecv(&u[local_n], 1, MPI_DOUBLE, rank + 1, 11,
                    &u[local_n + 1], 1, MPI_DOUBLE, rank + 1, 10,
                    MPI_COMM_WORLD, &status);
    }
}

static void compute_step_local(int rank, int nprocs, int start, int local_n,
                               const double *u, const double *u_prev, double *u_next) {
    const double coef = C * C * DT * DT / (DX * DX);
    (void)nprocs;

    for (int i = 1; i <= local_n; i++) {
        int g = start + i - 1;
        if (g <= 0 || g >= global_n - 1) {
            u_next[i] = u[i];
            continue;
        }
        u_next[i] = 2.0 * u[i] - u_prev[i]
                    + coef * (u[i + 1] - 2.0 * u[i] + u[i - 1]);
    }
}

static void write_output_parallel(int rank, int nprocs, int start, int local_n,
                                  const double *u, int step) {
    double *gather_buf = NULL;
    int *counts = NULL;
    int *displs = NULL;

    if (rank == 0) {
        gather_buf = (double *)malloc((size_t)global_n * sizeof(double));
        counts = (int *)malloc((size_t)nprocs * sizeof(int));
        displs = (int *)malloc((size_t)nprocs * sizeof(int));
    }

    for (int r = 0; r < nprocs; r++) {
        int s, ln;
```

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$ ls -la wave_mpi
-rwxr-xr-x 1 jovyan users 17000 May 20 04:01 wave_mpi
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$

jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$ mpirun -np 16 ./wave_mpi
WALLTIME 0.012256129
Simulation complete (N=200000, steps=500, io=0, nprocs=16).
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$

jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$ cat scaling_school_ex1.csv
nprocs,time_sec
1,0.312995405
2,0.079838486
4,0.047414820
8,0.027186643
16,0.035058569
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe1$
```

4. Strong-scaling results (time in seconds)

Fixed global problem size: **N = 200000**; **STEPS = 500**; process counts: **1, 2, 4, 8, 16**. I/O **disabled** (**WAVE_IO=0**) for timing.

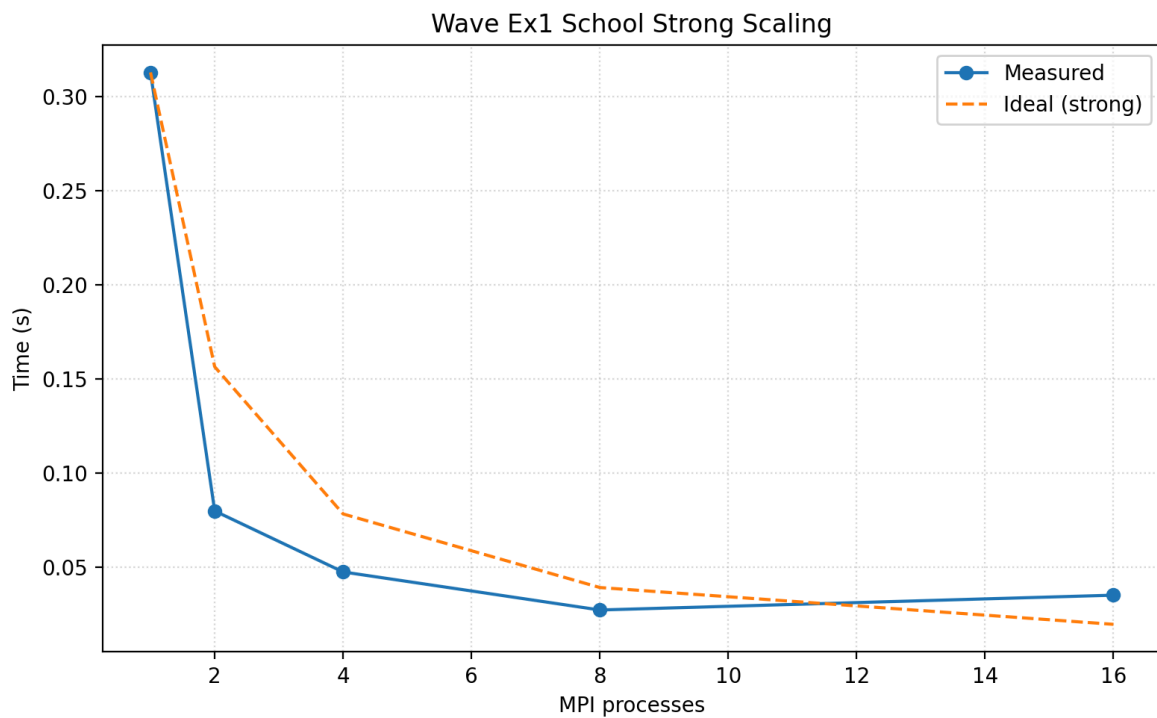
Processes	School Time (s)	School Speedup	Dardel Time (s)	Dardel Speedup
1	0.3130	1.0000	0.1447	1.0000
2	0.0798	3.92	0.0738	1.96
4	0.0474	6.60	0.0390	3.71
8	0.0272	11.51	0.0231	6.27
16	0.0351	8.93	0.0144	10.01

Speedup definition: $S(p) = T(1) / T(p)$.

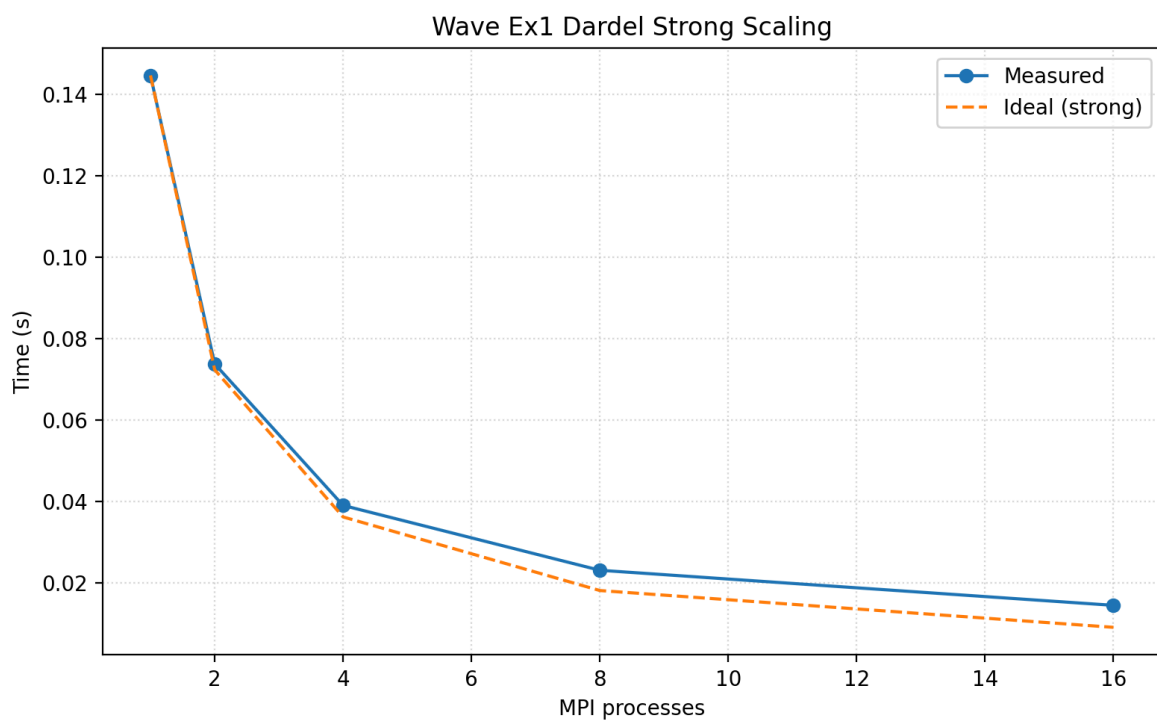
Raw CSV: [Exercise 1/scaling_school_ex1.csv](#), [Exercise 1/scaling_dardel_ex1.csv](#).

5. Plots: measured strong scaling + ideal scaling (dashed)

School cluster:



Dardel:



6. Performance analysis

School cluster:

- Runtime drops steadily from 1 to 8 processes ($0.313s \rightarrow 0.027s$, speedup **11.5×** at 8 ranks), showing good strong scaling while local work per rank is still large enough.
- At **16 processes**, time increases to $0.035s$ (speedup **8.9×** vs ideal 16×), indicating overhead dominates: halo latency per step, MPI startup/synchronization, and possible resource contention on the shared Jupyter/cluster node.
- Parallel efficiency at 16 ranks: $8.93/16 \approx 56\%$.

Dardel:

- Monotonic improvement through **16 ranks** (`0.145s` → `0.014s`, speedup **10.0×**), with no regression at the largest scale in this run.
- Scaling is smoother than school at high rank counts; job used up to **4 nodes × 4 MPI ranks/node** for `np=16` as required.
- Efficiency at 16 ranks: `10.01/16 ≈ 63%`; remaining gap to ideal is mainly halo communication and barriers each step.
- Dardel is faster than school at all process counts in this dataset (e.g. `T(1)=0.145s` vs `0.313s`), consistent with dedicated compute nodes vs shared interactive environment.

7. Score-P profiling (Dardel, largest scale: np = 16)

After the initial legacy-module failure (`PDCOLD/21.11` / `Score-P/7.0-cpeGNU-21.11`), we used the current Dardel-compatible module:

```
module load PDC/24.11 PrgEnv-cray cray-mpich/8.1.31 score-p/9.4-cpeCray-24.11
scorep --mpp=mpi --thread=none cc -O3 -O wave_mpi_scorep wave_mpi.c -lm
export SCOREP_ENABLE_TRACING=false SCOREP_ENABLE_PROFILING=true
export SCOREP_EXPERIMENT_DIRECTORY=scorep_ex1
srun -N 4 -n 16 --ntasks-per-node=4 ./wave_mpi_scorep
scorep-score -r scorep_ex1/profile.cubex
```

`scorep-score` summary (`np=16`, `WAVE_N=200000`, `WAVE_STEPS=500`, `WAVE_IO=0`):

Metric	Value
Total MPI calls (<code>visits</code> , aggregate)	15,096
Time spent in MPI calls	10.60 s (98.7% of total runtime 10.74 s)
Dominant MPI routines	<code>MPI_Init</code> (9.97 s), <code>MPI_Sendrecv</code> (15,000 calls), <code>MPI_Finalize</code> (0.43 s)

Exercise 2 - Parallel Row Sum Computation using MPI Collectives

1. Implementation

We parallelized row-sum computation in `Exercise 2/row_sum_mpi.c` using **MPI collective communication**.

Parallelization strategy

1. Rank `0` builds the full `N × N` matrix (or generates rows on the fly).
2. `MPI_Scatterv`: distribute matrix rows across ranks (variable row counts supported).
3. Local work: each rank computes partial row sums for its rows.
4. `MPI_Gatherv`: collect all row sums to rank `0` and write `row_sums_output.txt`.
5. `MPI_Reduce`: compute global sum of all matrix elements with `MPI_SUM` on rank `0`.

6. Timing: `MPI_Wtime()` + `MPI_Barrier` around scatter + local compute + gather/reduce.

`MPI_Scatterv` and `MPI_Gatherv` are used as generalized forms of `MPI_Scatter` / `MPI_Gather` to support uneven row distribution when `N` is not divisible by `P`.

Conceptual structure:

```
MPI_Scatter(matrix, rows_per_rank * N, MPI_DOUBLE,
           local_matrix, local_rows * N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

compute_row_sums(local_matrix, local_row_sums, local_rows);

MPI_Gather(local_row_sums, local_rows, MPI_DOUBLE,
          row_sums, local_rows, MPI_DOUBLE, 0, MPI_COMM_WORLD);

MPI_Reduce(&local_total, &global_total, 1, MPI_DOUBLE, MPI_SUM, 0,
          MPI_COMM_WORLD);
```

Weak-scaling setup

- **Weak scaling:** when `p` doubles, **global rows** double so each rank keeps ~constant row count.
- Example: base `N_base = 1000` rows at `p=1`; at `p=16`, global rows `N = 16000`.
- Implement via environment variable `WEAK_N` or command-line argument.

2. Build and run commands

School cluster

```
cd ~/dd2356_a4/Exe2
chmod +x run_school_ex2.sh run_scaling_weak.sh
./run_school_ex2.sh
```

Compile: `mpicc -O3 -o row_sum_mpi row_sum_mpi.c`

Weak scaling run: `mpirun -np <np> ./row_sum_mpi` with `WEAK_N=<1000*np>` and `ROWSUM_IO=0`.

Dardel

```
cd ~/DD2356/assignment4
source scripts/dardel_setup_env.sh
cd "Exercise 2"
cc -O3 -o row_sum_mpi row_sum_mpi.c

# Batch weak scaling (np = 1, 2, 4, 8, 16; N = 1000 * np)
sbatch --account=edu26.dd2356 scripts/dardel_ex2_scaling.slurm
```

Compile (Dardel): `cc -O3 -o row_sum_mpi row_sum_mpi.c` after loading `PDC/24.11`, `PrgEnv-gnu/8.6.0`, `cray-mpich/8.1.31`.

Run (Dardel, inside allocation): `srtn -n <np> ./row_sum_mpi` with `WEAK_N=<1000*np>` and `ROWSUM_IO=0` for timing runs.

3. Correctness validation

Compare `row_sums_output.txt` from serial and parallel runs:

```
python3 plot_row_sums.py
```

School cluster screenshots:

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$ sed -n '95,150p' row_sum_mpi.c
    fprintf(stderr, "Rank 0: failed to allocate matrix N=%d\n", global_n);
    MPI_Abort(MPI_COMM_WORLD, 1);
}
initialize_matrix_rows(full_matrix, 0, global_n, global_n);
}

local_matrix = (double *)malloc(local_elems * sizeof(double));
local_row_sums = (double *)malloc((size_t)local_rows * sizeof(double));
if (!local_matrix || !local_row_sums) {
    fprintf(stderr, "Rank %d: allocation failed\n", rank);
    MPI_Abort(MPI_COMM_WORLD, 1);
}

MPI_Barrier(MPI_COMM_WORLD);
t0 = MPI_Wtime();

{
    int *sendcounts = NULL;
    int *displs = NULL;
    if (rank == 0) {
        sendcounts = (int *)malloc((size_t)nprocs * sizeof(int));
        displs = (int *)malloc((size_t)nprocs * sizeof(int));
        for (int r = 0; r < nprocs; r++) {
            int lr = local_row_count(r, nprocs, global_n);
            sendcounts[r] = lr * global_n;
            displs[r] = row_start(r, nprocs, global_n) * global_n;
        }
    }
    MPI_Scatterv(full_matrix, sendcounts, displs, MPI_DOUBLE,
                local_matrix, (int)local_elems, MPI_DOUBLE,
                0, MPI_COMM_WORLD);
    if (rank == 0) {
        free(sendcounts);
        free(displs);
    }
}

compute_row_sums_local(local_matrix, local_row_sums, local_rows, global_n);
local_total = local_matrix_total(local_matrix, local_rows, global_n);

if (rank == 0) {
    all_row_sums = (double *)malloc((size_t)global_n * sizeof(double));
    if (!all_row_sums) MPI_Abort(MPI_COMM_WORLD, 1);
}

{
    int *recvcounts = NULL;
    int *displs = NULL;
    if (rank == 0) {
        recvcounts = (int *)malloc((size_t)nprocs * sizeof(int));
        displs = (int *)malloc((size_t)nprocs * sizeof(int));
        for (int r = 0; r < nprocs; r++) {
            recvcounts[r] = local_row_count(r, nprocs, global_n);
            displs[r] = row_start(r, nprocs, global_n);
        }
    }
}
```

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$ ls -la row_sum_mpi
-rwxr-xr-x 1 jovyan users 21072 May 20 04:14 row_sum_mpi
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$ █

jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$ mpirun -np 16 ./row_sum_mpi
WALLTIME 1.295849814
Row sum complete (N=16000, nprocs=16, io=0).
Global matrix sum (MPI Reduce): 2068350720000.000000
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$ █

jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$ cat scaling_weak_school_ex2.csv
nprocs,time_sec,N
1,0.005322549,1000
2,0.019240671,2000
4,0.091079259,4000
8,0.321056495,8000
16,1.619959236,16000
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe2$
```

Dardel screenshots (code / compile / run / scaling):

```
$ cd ~/DD2356/assignment4/Exercise 2
# row_sum_mpi.c: Scatterv / Gatherv / Reduce (excerpt)
```

```
{
    int *sendcounts = NULL;
    int *displs = NULL;
    if (rank == 0) {
        sendcounts = (int *)malloc((size_t)nprocs * sizeof(int));
        displs = (int *)malloc((size_t)nprocs * sizeof(int));
        for (int r = 0; r < nprocs; r++) {
            int lr = local_row_count(r, nprocs, global_n);
            sendcounts[r] = lr * global_n;
            displs[r] = row_start(r, nprocs, global_n) * global_n;
        }
    }
    MPI_Scatterv(full_matrix, sendcounts, displs, MPI_DOUBLE,
                local_matrix, (int)local_elems, MPI_DOUBLE,
                0, MPI_COMM_WORLD);
    if (rank == 0) {
        free(sendcounts);
        free(displs);
    }
}

compute_row_sums_local(local_matrix, local_row_sums, local_rows, global_n);
local_total = local_matrix_total(local_matrix, local_rows, global_n);

if (rank == 0) {
    all_row_sums = (double *)malloc((size_t)global_n * sizeof(double));
    if (!all_row_sums) MPI_Abort(MPI_COMM_WORLD, 1);
}

{
    int *recvcounts = NULL;
    int *displs = NULL;
    if (rank == 0) {
        recvcounts = (int *)malloc((size_t)nprocs * sizeof(int));
        displs = (int *)malloc((size_t)nprocs * sizeof(int));
        for (int r = 0; r < nprocs; r++) {
            recvcounts[r] = local_row_count(r, nprocs, global_n);
            displs[r] = row_start(r, nprocs, global_n);
        }
    }
    MPI_Gatherv(local_row_sums, local_rows, MPI_DOUBLE,
                all_row_sums, recvcounts, displs, MPI_DOUBLE,
                0, MPI_COMM_WORLD);
    if (rank == 0) {
        free(recvcounts);
        free(displs);
    }
}

MPI_Reduce(&local_total, &global_total, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

MPI_Barrier(MPI_COMM_WORLD);
t1 = MPI_Wtime();

if (rank == 0) {
    if (enable_io) write_output(all_row_sums, global_n);
    printf("WALLTIME %.9f\n", t1 - t0);
    printf("Row sum complete (N=%d, nprocs=%d, io=%d).\n",
           global_n, nprocs, enable_io);
    printf("Global matrix sum (MPI_Reduce): %.6f\n", global_total);
}
}
```

```
$ cd ~/DD2356/assignment4
$ source scripts/dardel_setup_env.sh
$ cd "Exercise 2"
$ cc -O3 -o row_sum_mpi row_sum_mpi.c
$ ls -la row_sum_mpi

cc: /opt/cray/pe/craype/2.7.33/bin/cc
Modules: PDC/24.11 PrgEnv-gnu/8.6.0 cray-mpich/8.1.31
-rwx--x--x 1 weiyil weiyil 16744 row_sum_mpi
```

```

$ export WEAK_N=16000 ROWSUM_IO=0
$ srun -n 16 ./row_sum_mpi

WALLTIME 0.122397189
Row sum complete (N=16000, nprocs=16, io=0).
Global matrix sum (MPI_Reduce): 2068350720000.000000

(Job: sbatch a4-ex2-rowsum, partition main, weak scaling np=16)

```

```
$ cat scaling_dardel_ex2.csv
```

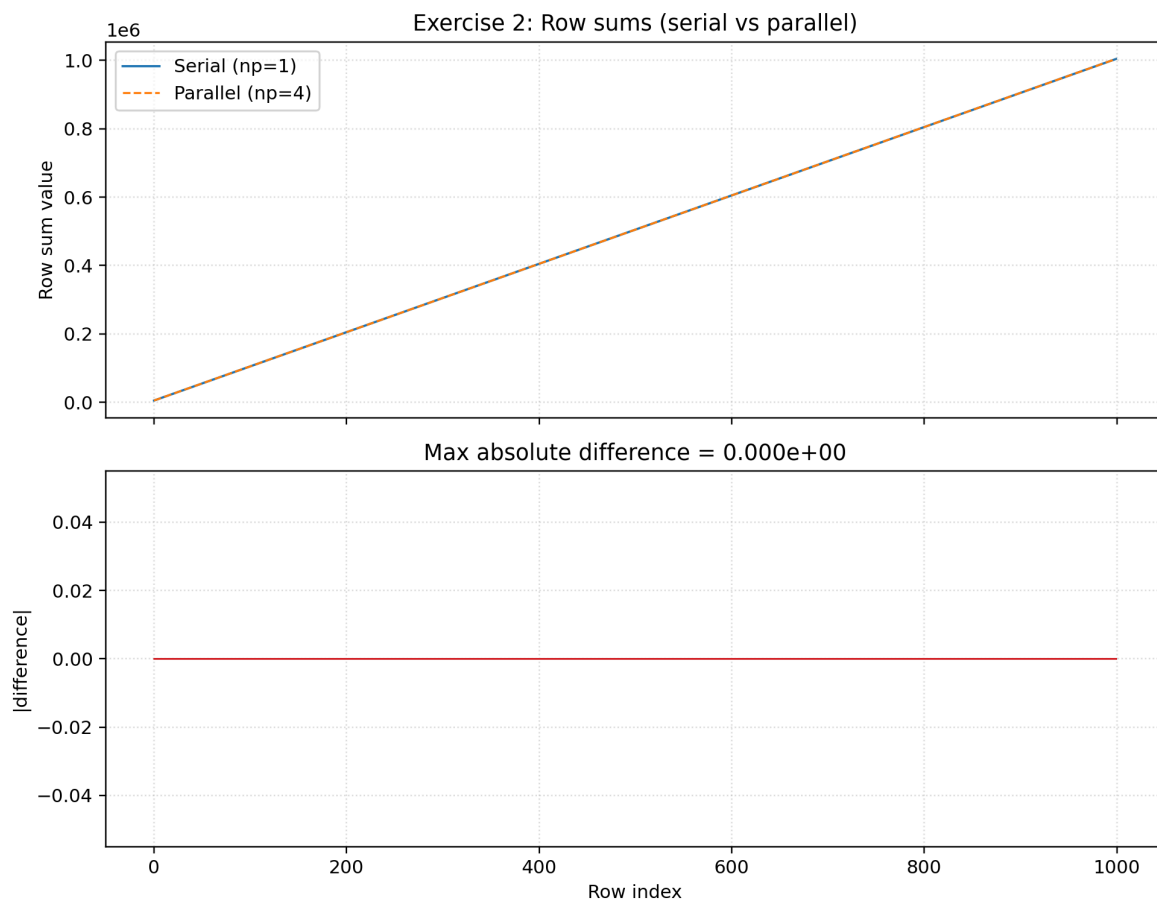
```

nprocs,time_sec,N
1,0.003427010,1000
2,0.012275642,2000
4,0.040387935,4000
8,0.090619604,8000
16,0.122397189,16000

```

At `np=16`, `MPI_Reduce` global sum = `2.06835072e12` (matches formula for initialized matrix; same on school and Dardel).

Serial vs parallel row-sum visualization (`WEAK_N=1000` , serial = `np=1` , parallel = `np=4`):



4. Weak-scaling results (time in seconds)

Base rows per rank: `N_base = 1000` (global `N = 1000 × p`); process counts: `1, 2, 4, 8, 16`.
I/O disabled for timing.

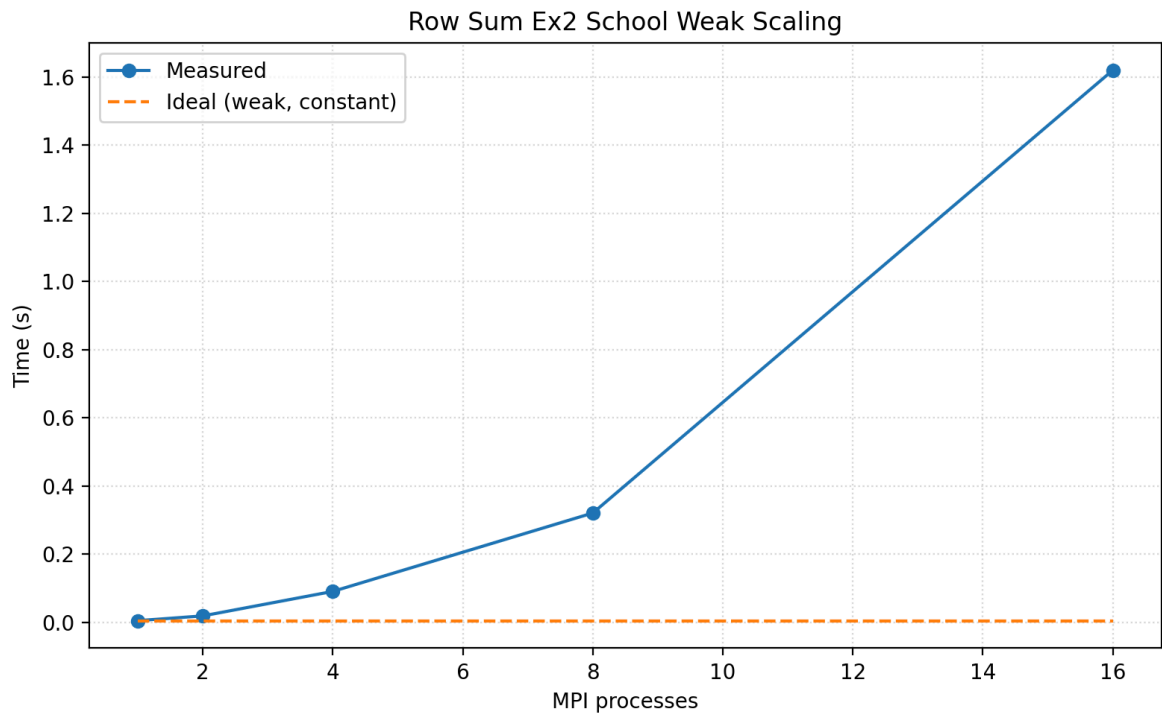
Processes	Global N	School Time (s)	Dardel Time (s)
1	1000	0.00532	0.00343
2	2000	0.01924	0.01228
4	4000	0.09108	0.04039
8	8000	0.32106	0.09062
16	16000	1.61996	0.12240

Raw CSV: `Exercise 2/scaling_weak_school_ex2.csv`, `Exercise 2/scaling_dardel_ex2.csv`
(copies in `Artifacts/results/dardel/`).

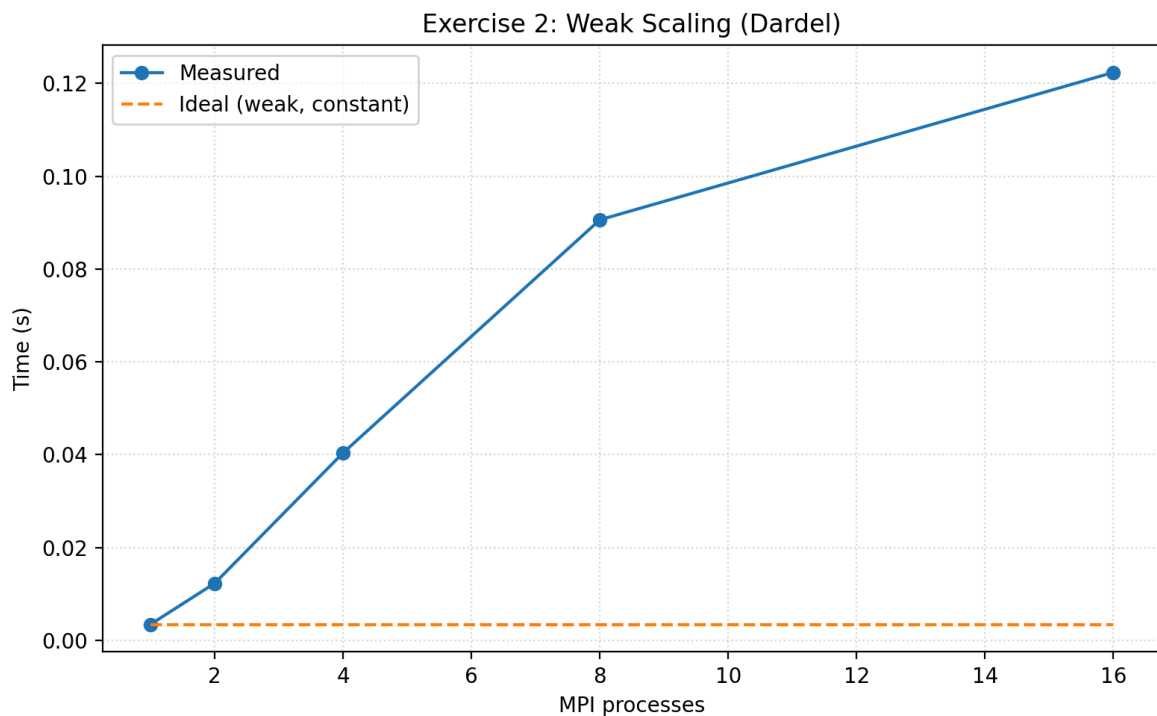
Ideal weak scaling: $T(p) \approx T(1)$.

5. Plots: measured weak scaling + ideal (flat) line

School cluster:



Dardel:



6. Performance analysis

School cluster:

- Weak scaling is **not flat**: time grows from 0.005s (1 proc) to 1.62s (16 procs) while per-rank row count stays ~1000.
- Main reason: rank 0 still builds the full $N \times N$ matrix ($O(N^2)$ memory and init), and `MPI_Scatterv` / `MPI_Gatherv` move $O(N^2)$ data each run. As N grows in weak scaling, these collective costs dominate.
- At `np=16`, communication and root allocation for `N=16000` (~2 GB matrix) explain the large increase.

Dardel:

- Time increases from 0.0034s to 0.122s ($\times 36$) vs school $\times 304$ for the same weak-scaling sequence.
- Dardel still shows overhead at large N (collectives + root $O(N^2)$ setup), but dedicated nodes and faster interconnect reduce the gap vs ideal weak scaling.
- At `np=16`, Dardel is **~13x faster** than school (0.122s vs 1.62s).
- Score-P at largest scale is reported in §7 below.

7. Score-P profiling (Dardel, largest scale: np = 16)

We profiled `row_sum_mpi` at largest scale (`np=16`, `WEAK_N=16000`, `ROWSUM_IO=0`) with `score-p/9.4-cpeCray-24.11` on the `PDC/24.11` stack.

Metric	Value
Total MPI calls (<code>visits</code> , aggregate)	144
Time spent in MPI calls	16.95 s (96.0% of total runtime 17.65 s)

Metric	Value
Dominant MPI routines	<code>MPI_Init</code> (10.17 s), <code>MPI_Barrier</code> (4.59 s), <code>MPI_Scatterv</code> (1.57 s)

Exercise 3 - 2D Game of Life with MPI and Non-Blocking Communication

1. Implementation

We parallelized Conway's Game of Life in `Exercise 3/gol_mpi.c` with **2D domain decomposition** and **non-blocking ghost-cell exchange**.

Parallelization strategy

- Factorize `P` processes into a 2D grid `Px × Py` (e.g., for `P=16`, use `4×4`).
- Each rank owns a subdomain with **one-cell-wide ghost layers** on all four sides.
- Each generation:
 - Post `MPI_Irecv` for ghost rows/columns from neighbors.
 - Post `MPI_Isend` of boundary data to neighbors.
 - Optionally compute interior cells while communication is in flight.
 - `MPI_Waitall` before updating boundary cells.
 - Apply Conway rules locally.
- Periodic boundaries: map neighbor ranks with modulo in x and y (torus).
- Timing: `MPI_Wtime()` + `MPI_Barrier` around the main loop; disable `write_output` for scalability runs.

Non-blocking halo pattern (conceptual):

```
MPI_Request reqs[8];
/* Irecv ghost rows/cols from up/down/left/right */
MPI_Irecv(..., &reqs[0]);
MPI_Isend(..., &reqs[1]);
/* ... */
update_interior();          /* overlap with communication */
MPI_Waitall(nreq, reqs, MPI_STATUSES_IGNORE);
update_boundaries();
```

Conway rules (unchanged)

- Live cell with `<2` or `>3` neighbors dies.
- Dead cell with exactly `3` neighbors becomes alive.
- Otherwise state is unchanged.

2. Build and run commands

School cluster (efficiency)

```
cd ~/dd2356_a4/Exe3
chmod +x run_school_ex3.sh run_efficiency.sh
./run_school_ex3.sh
```

Compile: `mpicc -O3 -o gol_mpi gol_mpi.c`

Run (efficiency): `mpirun -np <np> ./gol_mpi` with `GOL_N=2000 GOL_STEPS=500 GOL_IO=0`

3. Correctness validation and visualization

MPI uses **directional MPI tags** (`TAG_N2S`, `TAG_S2N`, etc.) so sender/receiver pairs match on every edge (fixes deadlock from mismatched tags).

School cluster screenshots:

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$ sed -n '135,200p' gol_mpi.c
static void exchange_ghosts(Grid *g, MPI_Comm cart, const int dims[2],
                           const int coords[2]) {
    int li = g->li, lj = g->lj, stride = g->stride;
    int rank_n, rank_s, rank_w, rank_e;
    MPI_Request reqs[24];
    int nreq = 0;

    /* Directional tags: sender and receiver of the same message use the SAME tag. */
    enum {
        TAG_N2S = 10, /* north -> south (north row sent to south ghost) */
        TAG_S2N = 11, /* south -> north */
        TAG_W2E = 20, /* west -> east */
        TAG_E2W = 21, /* east -> west */
        TAG_NW2SE = 30,
        TAG_SE2NW = 31,
        TAG_NE2SW = 32,
        TAG_SW2NE = 33
    };

    MPI_Cart_shift(cart, 0, 1, &rank_n, &rank_s);
    MPI_Cart_shift(cart, 1, 1, &rank_w, &rank_e);

    /* North neighbor sends row into my north ghost; I send row 1 southward for their ghost. */
    MPI_Irecv(&g->cur[0 * stride + 1], lj, MPI_INT, rank_n, TAG_N2S, cart, &reqs[nreq++]);
    MPI_Isend(&g->cur[1 * stride + 1], lj, MPI_INT, rank_n, TAG_S2N, cart, &reqs[nreq++]);

    MPI_Irecv(&g->cur[(li + 1) * stride + 1], lj, MPI_INT, rank_s, TAG_S2N, cart, &reqs[nreq++]);
    MPI_Isend(&g->cur[li * stride + 1], lj, MPI_INT, rank_s, TAG_N2S, cart, &reqs[nreq++]);

    MPI_Datatype col_type;
    MPI_Type_vector(li, 1, stride, MPI_INT, &col_type);
    MPI_Type_commit(&col_type);

    MPI_Irecv(&g->cur[1 * stride + 0], 1, col_type, rank_w, TAG_W2E, cart, &reqs[nreq++]);
    MPI_Isend(&g->cur[1 * stride + 1], 1, col_type, rank_w, TAG_E2W, cart, &reqs[nreq++]);

    MPI_Irecv(&g->cur[1 * stride + lj + 1], 1, col_type, rank_e, TAG_E2W, cart, &reqs[nreq++]);
    MPI_Isend(&g->cur[1 * stride + lj], 1, col_type, rank_e, TAG_W2E, cart, &reqs[nreq++]);

    if (li >= 2 && lj >= 2) {
```

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$ ls -la gol_mpi
-rwxr-xr-x 1 jovyan users 21624 May 20 05:12 gol_mpi
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$
```

```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$ mpirun -np 16 ./gol_mpi
WALLTIME 0.228579114
Game of Life complete (N=2000, steps=500, io=0, nprocs=16).
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$
```

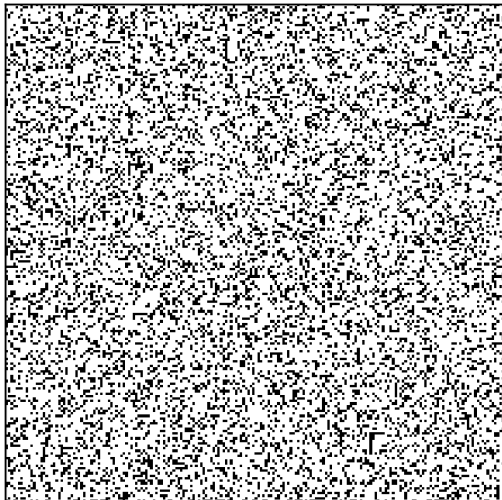


```
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$ cat efficiency_school_ex3.csv
nprocs,time_sec,speedup,efficiency
1,2.926551145,1.0,1.0
2,1.565721841,1.86914,0.93457
4,0.829065355,3.52994,0.882485
8,0.377071366,7.76127,0.970159
16,0.282264383,10.3681,0.648006
jovyan@jupyter-jinyeg:~/dd2356_a4/Exe3$
```

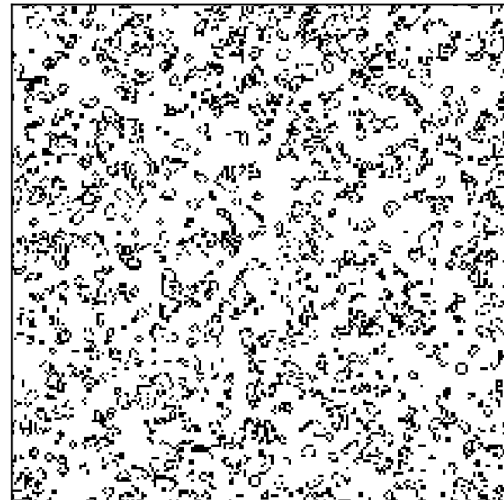
Game-of-Life correctness visualization (`GOL_N=200` , `GOL_STEPS=40` , `GOL_IO=1` , `np=4` ; snapshots at steps 0/10/20/30):

Exercise 3: Game of Life snapshots (correctness run, np=4)

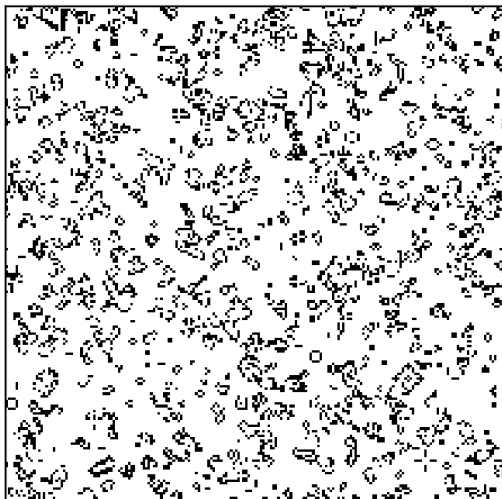
Step 0 (live=10076)



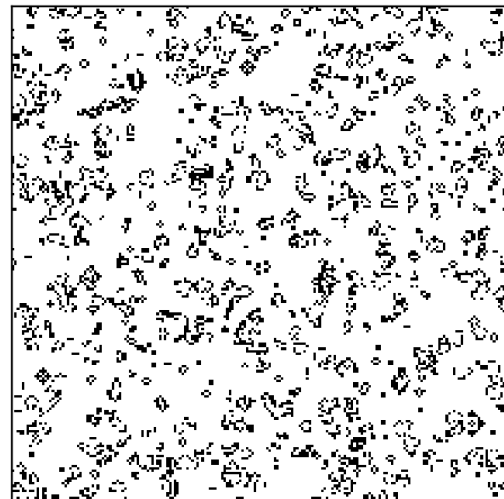
Step 10 (live=7602)



Step 20 (live=6388)



Step 30 (live=5511)



Visualize correctness with `python3 plot_gol.py` on `gol_output_*.txt` from a correctness run (`GOL_IO=1`).

4. Parallel efficiency (school cluster)

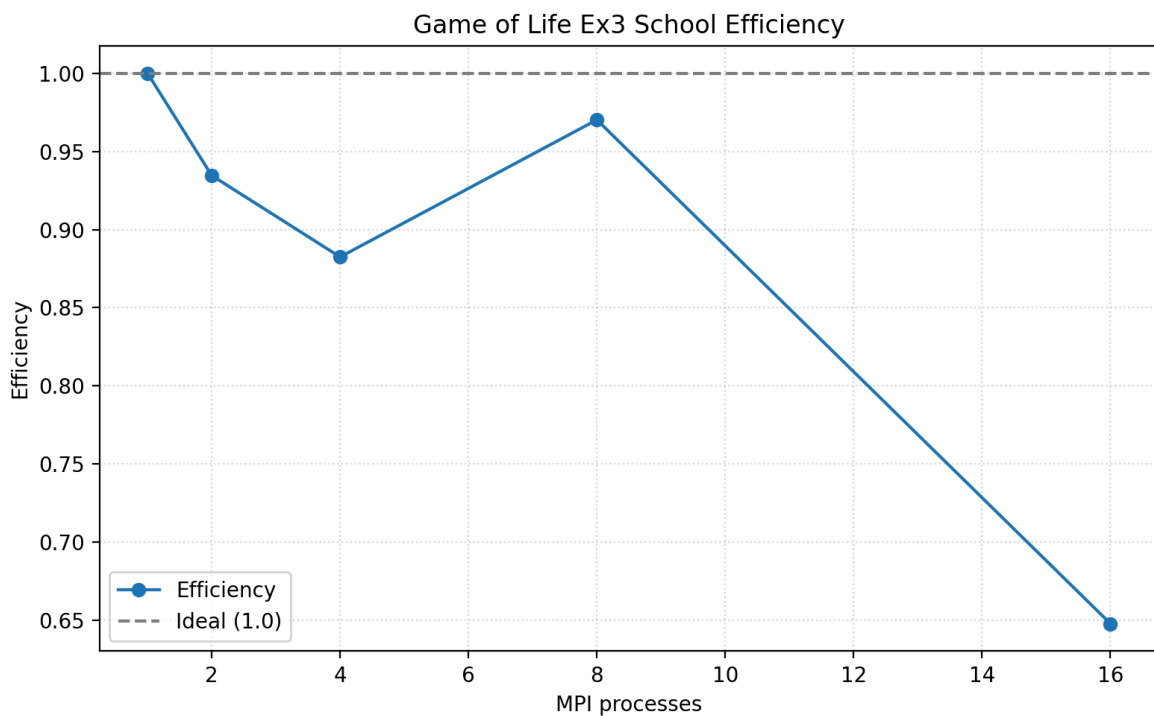
Problem size: `N = 2000` , `STEPS = 500` , `I/O disabled`.

Note: Figure 17b is a correctness visualization run (`N=200` , `STEPS=40` , `I/O=1`), while the efficiency table/plot below use the timing configuration (`N=2000` , `STEPS=500` , `I/O=0`).

Processes	Time (s)	Speedup $S(p)$	Efficiency $E(p)=S(p)/p$
1	2.9266	1.0000	1.0000
2	1.5657	1.8691	0.9346
4	0.8291	3.5299	0.8825
8	0.3771	7.7613	0.9702
16	0.2823	10.3681	0.6480

Raw CSV: `Exercise 3/efficiency_school_ex3.csv`.

5. Plot: parallel efficiency vs process count (school cluster)



6. Performance analysis (school cluster)

- Speedup grows to **10.37×** at 16 processes; efficiency stays **>88%** through 8 processes, then drops to **65%** at 16.
- The drop at 16 processes is typical when ghost communication and `MPI_Waitall` overhead grow relative to per-rank interior work, even with interior/computation overlap during edge exchange.
- Non-blocking `MPI_Irecv`/`MPI_Isend` allow `update_interior()` while edge messages are in flight; boundary cells are updated after the first `waitall` and corner exchange.

Appendix: Reproducibility Files

Core documentation:

- `README.md`: project overview and compile/run summary.

- `Exercise 1/README.md`, `Exercise 2/README.md`, `Exercise 3/README.md`: per-exercise instructions.
- `scripts/README-dardel.md`: Dardel modules, `edu26.dd2356`, Slurm examples.

Source code:

- `Exercise 1/wave_mpi.c` — 1D wave + halo exchange
- `Exercise 2/row_sum_mpi.c` — row sums + collectives
- `Exercise 3/gol_mpi.c` — Game of Life + non-blocking ghosts

Scripts:

- `Exercise 1/run_scaling_strong.sh`, `Exercise 1/plot_wave.py`
- `Exercise 2/run_scaling_weak.sh`, `Exercise 2/plot_row_sums.py`
- `Exercise 3/run_efficiency.sh`, `Exercise 3/plot_gol.py`
- `plot_scaling_mpi.py` — strong/weak scaling plots
- `scripts/dardel_setup_env.sh`, `scripts/dardel_run_mpi.slurm`

Results and plots:

- Exercise 1 (completed):
 - `Exercise 1/scaling_school_ex1.csv`, `Exercise 1/scaling_dardel_ex1.csv`
 - `Artifacts/plots/school/strong_scaling_ex1_school.png`
 - `Artifacts/plots/school/wave_serial_vs_parallel_ex1.png`
 - `Artifacts/plots/dardel/strong_scaling_ex1_dardel.png`
- Exercise 2 (completed):
 - `Exercise 2/scaling_weak_school_ex2.csv`, `Exercise 2/scaling_dardel_ex2.csv`
 - `Artifacts/plots/school/weak_scaling_ex2_school.png`
 - `Artifacts/plots/school/row_sums_serial_vs_parallel_ex2.png`
 - `Artifacts/plots/dardel/weak_scaling_ex2_dardel.png`
 - `Artifacts/screenshots/school/ex2_*.png`
 - `Artifacts/screenshots/dardel/ex2_*.png`
- Exercise 3 (school efficiency, per assignment):
 - `Exercise 3/efficiency_school_ex3.csv`
 - `Artifacts/plots/school/efficiency_ex3_school.png`
 - `Artifacts/plots/school/gol_visualization_ex3.png`
 - `Artifacts/screenshots/school/ex3_*.png`
- Score-P (Ex1-2, Dardel): completed with `score-p/9.4-cpeCray-24.11`; metrics reported in Exercise 1/2 §7
- Screenshots: `Artifacts/screenshots/school/`, `Artifacts/screenshots/dardel/` (Ex1-2)

Dardel project account: `edu26.dd2356`

Working directory on Dardel: `~/DD2356/assignment4`